Phase 3

Team: 5 | **Assigned Phase 2 submission converted:** Team 14 (EER model of Team 18's Speedrun Leaderboard requirements)

2A) ER-to-Relational Mapping

Step 1 : Mapping regular (strong) entities

Each regular entity type in the EER diagram is mapped to a separate relation containing its simple attributes. Composite attributes are replaced by their constituent simple attributes, while multivalued attributes are mapped to separate relations.

The primary key of each relation is selected from the key attributes of the corresponding entity. Foreign keys are introduced wherever required to preserve relationships between entities.

The composite location attribute of USER is represented by country_code and region_name. Similarly, the composite title attribute of GAME is represented by title_international and title_local.

PK = primary key; FK = foreign key; ? = optional.

- **USER(user_id, username, display_name, joined_at, is_admin, active, country_code?, region_name?)**
  PK: user_id, unique: username. Composite location is flattened.
- **GAME(game_id, slug, title_international, title_local?, release_year?, source_url?, active)**
  PK: game_id, unique: slug. Composite title is flattened.
- **CATEGORY(category_id, name, kind, active, game_id)**
  PK: category_id

  FK: game_id → GAME(game_id).
- **LEVEL(level_id, name, display_order, game_id)**
  PK: level_id

  FK: game_id → GAME(game_id).
- **PLATFORM(platform_id, name, manufacturer?)**
  PK: platform_id, unique: name.
- **LEADERBOARD(board_id, board_code, variant_code, variant_description, active, category_id, platform_id, level_id?)**
  PK: board_id, unique: board_code

FKs: category_id → CATEGORY, platform_id → PLATFORM, level_id → LEVEL.

- **RUN(run_id, run_date, submitted_at, duration_ms, note?, runner_id, submitter_id, board_id, version_no)**

  PK: run_id

  FKs: runner_id and submitter_id → USER, (board_id, version_no) → BOARD_RULE_VERSION.

- **PERMISSION(permission_code, description)**

  PK: permission_code.

Step 2 : Map weak entities

Each weak entity is mapped to a separate relation. Its primary key is formed by combining the primary key of its owner entity with its partial key.

The identifying relationship is represented through a foreign key referencing the owner relation.

The created_at attribute of BOARD_RULE_VERSION is retained from the correction introduced during Phase 2.

- **BOARD_RULE_VERSION(board_id, version_no, rules_text, timing_method, effective_at, created_at)**

  PK: (board_id, version_no)

  FK: board_id → LEADERBOARD.

- **EVIDENCE_LINK(run_id, evidence_no, URL, kind, label?)**

  PK: (run_id, evidence_no)

  FK: run_id → RUN.

- **REVIEW_EVENT(run_id, event_no, event_at, outcome, note?, author_id)**

  PK: (run_id, event_no)

  FKs: run_id → RUN, author_id → USER.

Step 3 : Map 1:N relationships

For each 1:N relationship, the primary key of the entity on the 1-side is included as a foreign key in the relation corresponding to the N-side.

These foreign keys have already been incorporated into the relations defined in Steps 1 and 2. Therefore, no additional relations are required for these relationships

Relationship	1 → N	Foreign key
DEFINES	GAME → CATEGORY	CATEGORY.game_id
CONTAINS	GAME → LEVEL	LEVEL.game_id
ORGANIZES	CATEGORY → LEADERBOARD	LEADERBOARD.category_id
HOSTS	PLATFORM → LEADERBOARD	LEADERBOARD.platform_id
SCOPES	LEVEL → LEADERBOARD	LEADERBOARD.level_id (nullable)
OWNS_RULES	LEADERBOARD → BOARD_RULE_VERSION	BOARD_RULE_VERSION.board_id
GOVERNS	BOARD_RULE_VERSION → RUN	RUN.(board_id, version_no)
PERFORMS	USER → RUN	RUN.runner_id
SUBMITS	USER → RUN	RUN.submitter_id
HAS_EVIDENCE	RUN → EVIDENCE_LINK	EVIDENCE_LINK.run_id
HAS_REVIEW	RUN → REVIEW_EVENT	REVIEW_EVENT.run_id
AUTHORS	USER → REVIEW_EVENT	REVIEW_EVENT.author_id

Step 4 : Mapping M:N and recursive relationships

Each M:N relationship is mapped to a separate relation containing the primary keys of the participating entities as foreign keys.

The SUPPORTS relation represents the many-to-many relationship between GAME and PLATFORM.

The RELATED_TO relation represents a recursive relationship between instances of GAME. The attributes source_game_id and related_game_id distinguish the two participating roles. Self-referential relationships are prohibited, and the relationship is treated as directed.

- **SUPPORTS(game_id, platform_id)**

  PK: (game_id, platform_id)

  FKs to GAME and PLATFORM. M:N GAME- PLATFORM relationship.

- **RELATED_TO(source_game_id, related_game_id)**

  PK: (source_game_id, related_game_id)

  both FKs → GAME(game_id). Directed recursive relationship; disallow self-links.

Step 5 : Map multi-valued attributes

Each multivalued attribute is represented using a separate relation containing the primary key of its owner entity and the components of the multivalued attribute.

GAME_ALTERNATE_NAME stores individual alternate names associated with each game. Option (b), separate relation: the number of alternate names per game is not fixed, so fixed columns in GAME are not suitable.

USER_SOCIAL_PROFILE stores the service and URL associated with each social profile of a user. The chosen composite primary key permits multiple URLs for the same service. Option (b), separate relation: the number of profiles per user is not fixed and each profile has its own sub-attributes (service, URL).

This mapping prevents multiple values from being stored within a single attribute.

- **GAME_ALTERNATE_NAME(game_id, alternate_name)**

  PK: (game_id, alternate_name)

  FK: game_id → GAME. Each alternate name occupies its own row.

- **USER_SOCIAL_PROFILE(user_id, service, URL)**

  PK: (user_id, service, URL)

  FK: user_id → USER.

Step 6 : Map ternary relationship

The ternary relationship AUTHORIZES, involving USER, GAME, and PERMISSION, is represented by a separate relation.

The primary keys of the three participating entities are included as foreign keys, and their combination forms the primary key of AUTHORIZES.

This preserves the association between a specific user, game, and permission without losing the meaning of the original ternary relationship.

- **AUTHORIZES(user_id, game_id, permission_code)**

  PK: (user_id, game_id, permission_code)

  FKs → USER, GAME, and PERMISSION.

Step 7 : Map specialization

The RUN entity is specialized into FULL_GAME_RUN and LEVEL_RUN. The specialization is total and disjoint.

According to the Phase 2 EER model, subtype membership is determined by the kind attribute of the CATEGORY associated with the run's leaderboard.

Option used: a single relation for the superclass with no subclass relations (textbook option 8C, single relation with a type attribute), except that the type is not stored in RUN and is derived from CATEGORY.kind instead. Since the subclasses have no additional attributes or relationships, the mapping retains a single RUN relation. Subtype membership is determined through the associated CATEGORY rather than through additional subtype relations.

This design avoids storing redundant discriminator information. The total and disjoint specialization constraints are enforced by ensuring that every RUN references a valid leaderboard whose associated CATEGORY has exactly one defined kind.

Step 8 : Handling derived attributes

Derived attributes are not stored directly in the relational schema because their values can be computed from existing data.

The current_status of a run is determined by its latest review event, with new used when no review event exists.

The board_rank of a run is determined from eligible verified personal-best performances on the corresponding leaderboard.

The record_duration_ms of a leaderboard is determined from the minimum eligible personal-best duration.

These values may be obtained through database queries or views.

Derived attribute	**Treatment**
RUN.current_status	Compute from latest REVIEW_EVENT.outcome; new if no review event.
RUN.board_rank	Compute using eligible verified personal bests per leaderboard.
LEADERBOARD.record_duration_ms	Compute from the minimum eligible personal-best duration.

Step 9: Verification and Integrity Constraints

The resulting relational schema contains 16 relations, consisting of eight relations corresponding to strong entities, three weak-entity relations, two M:N or recursive relationship relations, two multivalued-attribute relations, and one ternary relationship relation.

The 1:N relationships are represented through foreign keys, while the specialization of RUN does not introduce additional stored relations.

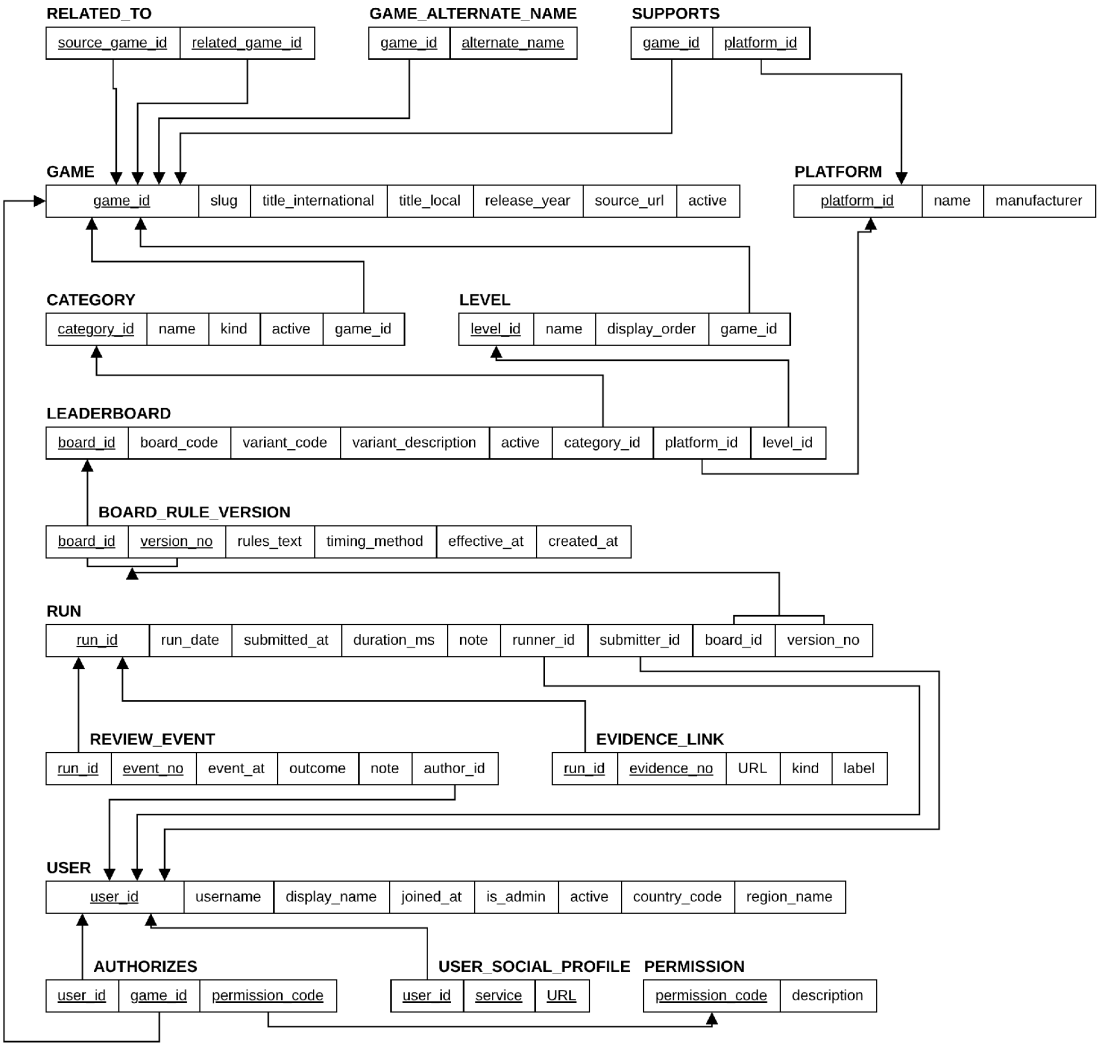
The relational schema preserves the following integrity constraints specified in the EER model:

- **Uniqueness:** CATEGORY(game_id, name, kind), LEVEL(game_id, name), and LEVEL(game_id, display_order) must be unique.
- **Game consistency:** A leaderboard's CATEGORY and optional LEVEL must belong to the same GAME.
- **Platform compatibility:** A leaderboard's PLATFORM must be supported by its associated GAME through the SUPPORTS relationship.
- **Specialization:** CATEGORY.kind = per_level if and only if LEADERBOARD.level_id is present; CATEGORY.kind = full_game if and only if LEADERBOARD.level_id is absent.
- **Evidence constraints:** Each RUN must have between 1 and 10 EVIDENCE_LINK records.
- **Review authorization:** A reviewer making a verified or rejected decision must possess the REVIEW_RUNS permission for the corresponding game and must not be the runner or submitter.
- **Recursive relationship:** RELATED_TO must not contain self-referential relationships, and the relationship is directed.
- **Rule-version chronology:** BOARD_RULE_VERSION.effective_at must not precede its created_at or the effective_at of the previous version.

Certain constraints cannot be enforced using primary keys and foreign keys alone. Additional mechanisms, such as database triggers, assertions where supported, or application-level validation, are required to enforce these business rules.

Notation: all snapshots use the Elmasri & Navathe relational schema notation (primary keys underlined, arrows from each foreign key to the key it references). One deviation: a composite foreign key is drawn as a single arrow between brackets that join the key's attributes.

Stage A snapshot



Link:

https://viewer.diagrams.net/?highlight=0000ff&nav=1&title=Stage_A.drawio#Uhttps%3A%2F%2Fdrive.google.com%2Fuc%3Fid%3D1YU7IOlMTzyjK_60LcKND4UDwxW6PwKJc%26export%3Ddownload

3.1) STAGE B: First Normal Form (1NF)

One relation is not in 1NF. For a correction event, REVIEW_EVENT.note must record the old and new value of every changed field: "the prior duration and all changed evidence fields, including additions/removals, using a validated structured format" (Team 18, p.15). Team 14 identifies the same hidden entity (C-3). A list of values of variable length in one column is a repeating group, so REVIEW_EVENT violates 1NF.

The changed values move to a new relation with one row per changed field. REVIEW_EVENT.note keeps only the explanation or reason.

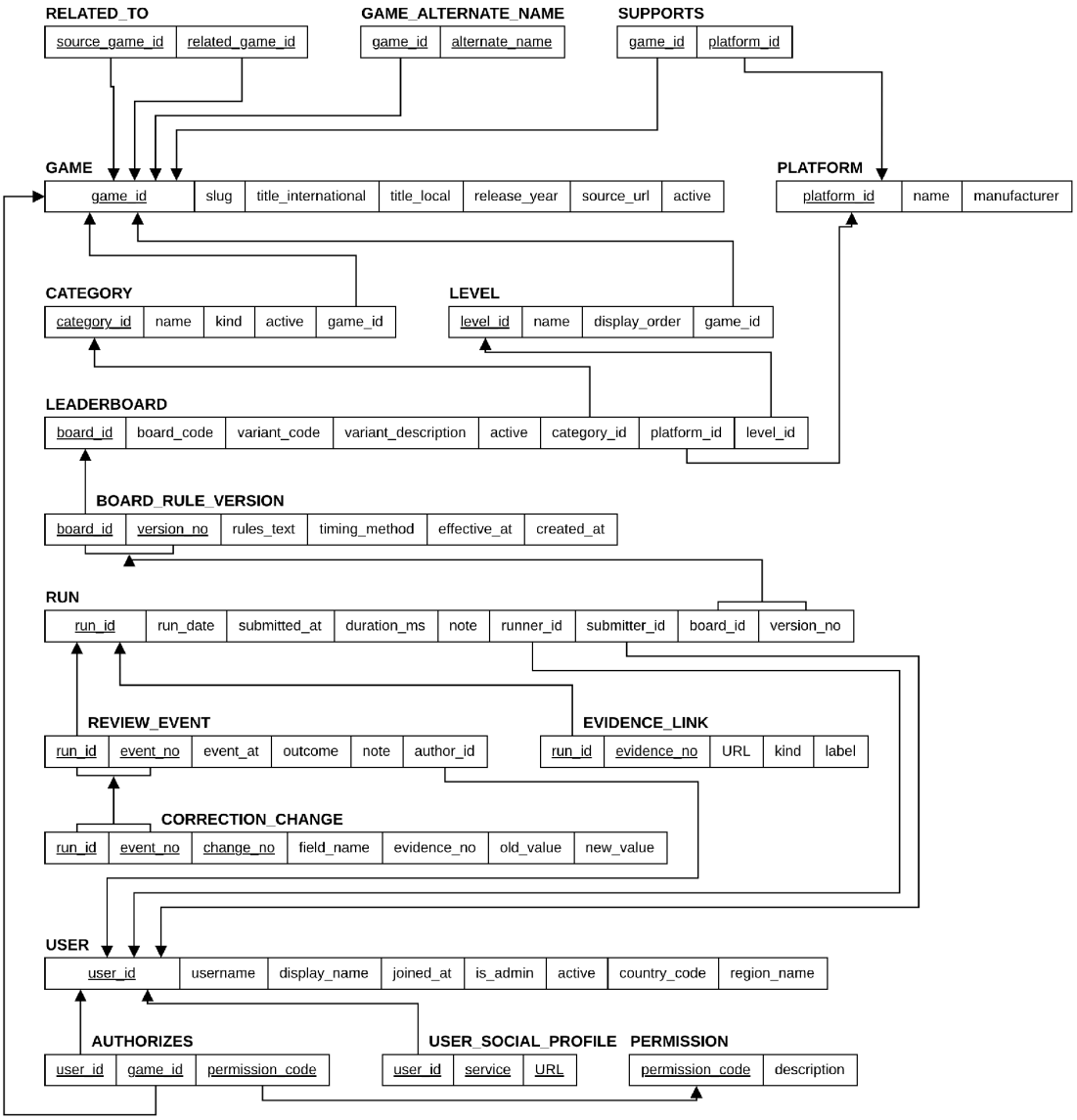
- **REVIEW_EVENT(run_id, event_no, event_at, outcome, note?, author_id)**
  PK: (run_id, event_no). FKs: run_id → RUN, author_id → USER.
- **CORRECTION_CHANGE(run_id, event_no, change_no, field_name, evidence_no?, old_value?, new_value?)**
  PK: (run_id, event_no, change_no). FK: (run_id, event_no) → REVIEW_EVENT.

field_name is duration_ms, URL, kind or label. evidence_no is empty for a duration change; old_value is empty for an added link and new_value for a removed link. evidence_no is not a foreign key, because a removed link no longer exists in EVIDENCE_LINK (Team 18, p.12: evidence snapshots are audit context, not current values).

All other relations already satisfy 1NF. The composite attributes location and title were flattened in Stage A, the multivalued attributes became GAME_ALTERNATE_NAME and USER_SOCIAL_PROFILE, and every other column holds a single value. The Stage B model has 17 relations.

Conversion performed: REVIEW_EVENT split; new relation CORRECTION_CHANGE.

Stage B snapshot



Link:

https://viewer.diagrams.net/?highlight=0000ff&nav=1&title=Stage_B.drawio#Uhttps%3A%2F%2Fdrive.google.com%2Fuc%3Fid%3D1bhaL1tGcdWREAqG65UunUzshFo2iIcaV%26export%3Ddownload

3.2) STAGE C: Second Normal Form (2NF)

One relation has a partial dependency. In BOARD_RULE_VERSION (key: board_id, version_no), timing_method depends on board_id alone: "Changed timing method or variant restrictions require a new board" (Team 18, p.8; also U3, p.12), so every version of a board has the same timing method. board_id → timing_method is a partial dependency of a non-prime attribute.

timing_method is removed from BOARD_RULE_VERSION. The relation (board_id, timing_method) that this creates has the key board_id, the same as LEADERBOARD, so it is merged into LEADERBOARD. The Stage C model has 17 relations.

- **BOARD_RULE_VERSION(board_id, version_no, rules_text, effective_at, created_at)**

  PK: (board_id, version_no). FK: board_id → LEADERBOARD.

- **LEADERBOARD(board_id, board_code, variant_code, variant_description, timing_method, active, category_id, platform_id, level_id?)**

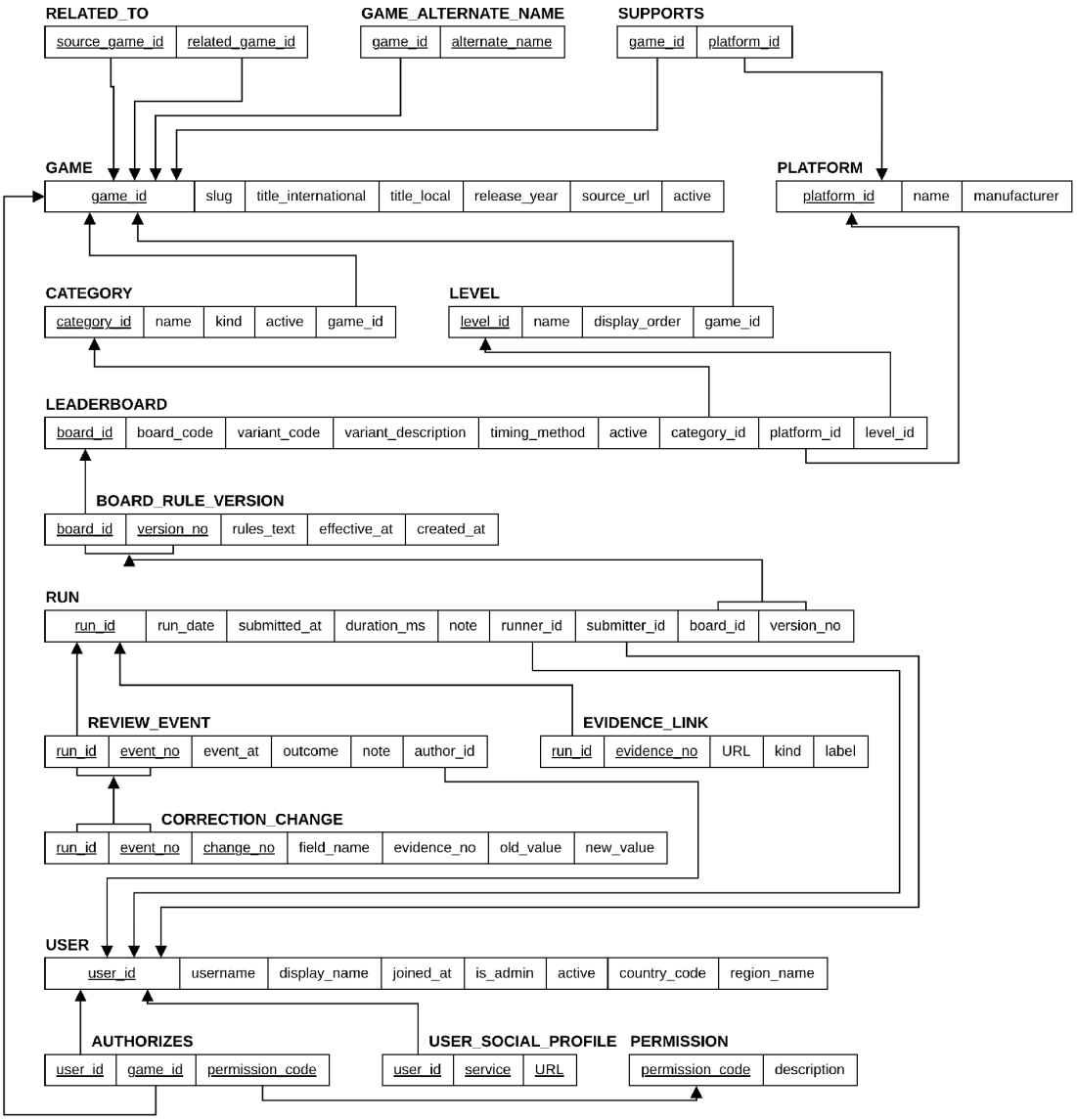
  PK: board_id; unique: board_code; unique: (category_id, level_id, platform_id, variant_code). FKs unchanged.

The other relations were checked against every candidate key:

- USER, GAME, PLATFORM, RUN and PERMISSION: every candidate key is a single attribute, so no partial dependency is possible.
- LEADERBOARD (keys: board_id; board_code; category_id, level_id, platform_id, variant_code): variant_description, timing_method and active describe one board and do not depend on part of the composite key.
- CATEGORY (keys: category_id; game_id, name, kind): active depends on the whole composite key. LEVEL (keys: level_id; game_id, name; game_id, display_order): every attribute is prime.
- BOARD_RULE_VERSION (after the change), EVIDENCE_LINK (keys: run_id, evidence_no; run_id, URL), REVIEW_EVENT (key: run_id, event_no) and CORRECTION_CHANGE (key: run_id, event_no, change_no): the other attributes describe one version, link, event or change, so they need the whole key.
- SUPPORTS, RELATED_TO, AUTHORIZES, GAME_ALTERNATE_NAME and USER_SOCIAL_PROFILE consist only of key attributes.

Conversion performed: timing_method moved from BOARD_RULE_VERSION to LEADERBOARD.

Stage C snapshot



Link:

https://viewer.diagrams.net/?highlight=0000ff&nav=1&title=Stage_C.drawio#Uhttps%3A%2F%2Fdrive.google.com%2Fuc%3Fid%3D19YWJJB9ixlb2xFzp96paxpv5__5bSXR1%26export%3Ddownload

3.3) STAGE D: Third Normal Form (3NF)

One relation has a transitive dependency. In RUN (key: run_id), version_no is fixed by the board and the submission time: "A run selects the latest effective version at its submission instant; its selected reference never changes" (Team 18, p.15), and constraint C8 (effective_at ≥ created_at) stops a later version from applying to an earlier submission. So board_id, submitted_at → version_no holds, where (board_id, submitted_at) is not a superkey and version_no is non-prime.

RUN is decomposed into two relations. The shared attributes (submitted_at, board_id) form the key of RUN_RULE, so the decomposition is lossless, and every run still reaches its rule version through GOVERNS (R8).

- **RUN(run_id, run_date, duration_ms, note?, runner_id, submitter_id, submitted_at, board_id)**

  PK: run_id. FKs: runner_id and submitter_id → USER, (submitted_at, board_id) → RUN_RULE.

- **RUN_RULE(submitted_at, board_id, version_no)**

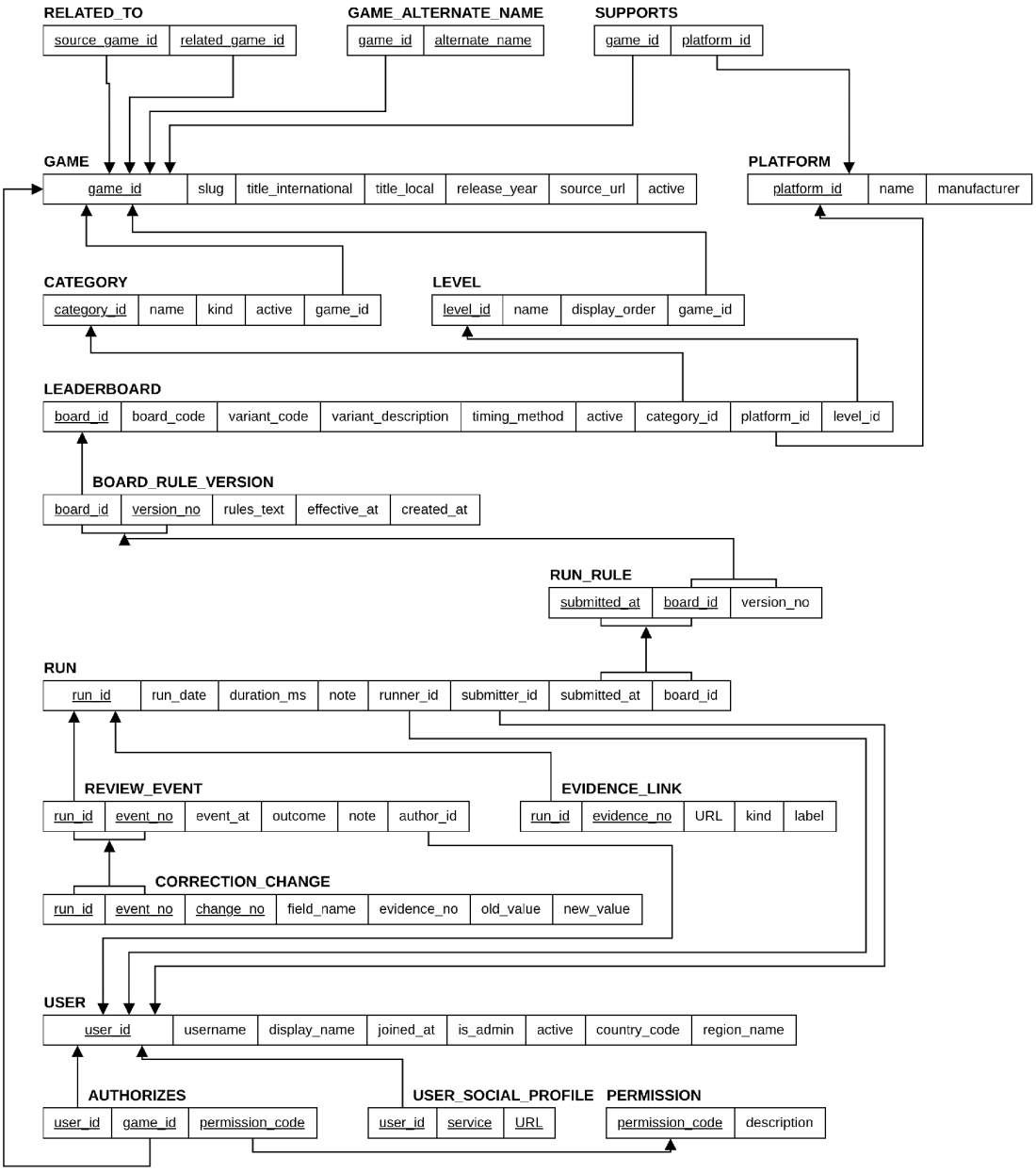
  PK: (submitted_at, board_id). FK: (board_id, version_no) → BOARD_RULE_VERSION.

The other relations were checked, and every determinant of a non-trivial dependency is a superkey:

- USER, GAME, CATEGORY, LEVEL, PLATFORM and PERMISSION: only candidate keys determine other attributes. In USER, region_name does not determine country_code, because a region name can occur in more than one country.
- LEADERBOARD: variant_description belongs to the board, not to variant_code. Team 18 defines a variant only as part of a board (p.4: "a board specifies the variant").
- BOARD_RULE_VERSION, REVIEW_EVENT and CORRECTION_CHANGE: only their candidate keys are determinants.
- EVIDENCE_LINK: kind is entered for each link and can be corrected on its own (U2, p.12), so URL does not determine kind.
- SUPPORTS, RELATED_TO, AUTHORIZES, GAME_ALTERNATE_NAME and USER_SOCIAL_PROFILE have no non-prime attributes. A social profile's service comes from a configured list (p.6), not from its URL.

Conversion performed: RUN split into RUN and RUN_RULE. The Stage D model has 18 relations, all in 3NF.

Stage D snapshot



Link:

https://viewer.diagrams.net/?highlight=0000ff&nav=1&title=Stage_D.drawio#Uhttps%3A%2F%2Fdrive.google.com%2Fuc%3Fid%3D1M3vdEBTywoAl4DLSMef-5j4f-7hQHgvk%26export%3Ddownload

3.4) Functional Dependencies (FDs)

Candidate keys and functional dependencies of every relation, written as A, B → C. Relations changed during normalization come first; the others were checked and left unchanged.

1. Relations changed during normalization

- **REVIEW_EVENT (Stage B)**

  Candidate key: {run_id, event_no}

  run_id, event_no → event_at, outcome, note, author_id

  Change: the old/new values held in note were a repeating group (1NF); they moved to CORRECTION_CHANGE.

- **CORRECTION_CHANGE (new in Stage B)**

  Candidate key: {run_id, event_no, change_no}

  run_id, event_no, change_no → field_name, evidence_no, old_value, new_value

- **BOARD_RULE_VERSION (Stage C)**

  Candidate key: {board_id, version_no}

  Before: board_id, version_no → rules_text, timing_method, effective_at, created_at; board_id → timing_method (partial dependency)

  After: board_id, version_no → rules_text, effective_at, created_at

- **LEADERBOARD (Stage C)**

  Candidate keys: {board_id}; {board_code}; {category_id, level_id, platform_id, variant_code} (an absent level counts as one full-game value, Team 18 p.8)

  board_id → board_code, variant_code, variant_description, timing_method, active, category_id, platform_id, level_id

  board_code → board_id

  category_id, level_id, platform_id, variant_code → board_id

- **RUN (Stage D)**

  Candidate key: {run_id}

  Before: run_id → run_date, submitted_at, duration_ms, note, runner_id, submitter_id, board_id, version_no; board_id, submitted_at → version_no (transitive dependency)

  After: run_id → run_date, duration_ms, note, runner_id, submitter_id, submitted_at, board_id

- **RUN_RULE (new in Stage D)**

Candidate key: {submitted_at, board_id}

submitted_at, board_id → version_no

2. Relations checked and left unchanged

- **USER**

  Candidate keys: {user_id}; {username}

  user_id → username, display_name, joined_at, is_admin, active, country_code, region_name

  username → user_id

  Checked: region_name → country_code does not hold.

- **GAME**

  Candidate keys: {game_id}; {slug}

  game_id → slug, title_international, title_local, release_year, source_url, active

  slug → game_id

- **CATEGORY**

  Candidate keys: {category_id}; {game_id, name, kind}

  category_id → name, kind, active, game_id

  game_id, name, kind → category_id

- **LEVEL**

  Candidate keys: {level_id}; {game_id, name}; {game_id, display_order}

  level_id → name, display_order, game_id

  game_id, name → level_id

  game_id, display_order → level_id

- **PLATFORM**

  Candidate keys: {platform_id}; {name}

  platform_id → name, manufacturer

  name → platform_id

- **PERMISSION**

  Candidate key: {permission_code}

  permission_code → description

- **EVIDENCE_LINK**

  Candidate keys: {run_id, evidence_no}; {run_id, URL} (Team 18, I2: duplicate evidence URLs are rejected)

run_id, evidence_no → URL, kind, label

run_id, URL → evidence_no

Checked: URL → kind does not hold.

- **SUPPORTS, RELATED_TO, AUTHORIZES, GAME_ALTERNATE_NAME, USER_SOCIAL_PROFILE**

  Candidate keys: {game_id, platform_id}; {source_game_id, related_game_id}; {user_id, game_id, permission_code}; {game_id, alternate_name}; {user_id, service, URL}

  Each key is the whole relation, so there are no non-trivial dependencies. Checked: URL → service does not hold in USER_SOCIAL_PROFILE.

3. Normalization Conclusion

Stage B moved the old/new values out of REVIEW_EVENT.note into CORRECTION_CHANGE (1NF). Stage C moved timing_method from BOARD_RULE_VERSION to LEADERBOARD (2NF). Stage D split RUN into RUN and RUN_RULE (3NF). The final model has 18 relations, all in 3NF.

4) Changes to the Assigned EER Model

No changes were made to the assigned EER diagram. The mapping uses its final Phase 2 version, including the created_at attribute that was added to BOARD_RULE_VERSION as a Phase 2 correction.

Flattening the composite attributes, using separate relations for the multi-valued attributes, keeping a single RUN relation for the specialization and not storing the derived attributes are mapping choices explained in Stage A, not changes to the diagram.

The decompositions in Stages B to D are normalization steps on the relational model, not changes to the EER diagram.

5) GenAI Usage Statement

We used AI to check the validity of our normalization result, and whether there were any inconsistencies in our document in general. We also used it to check whether the diagram made is correct or not.